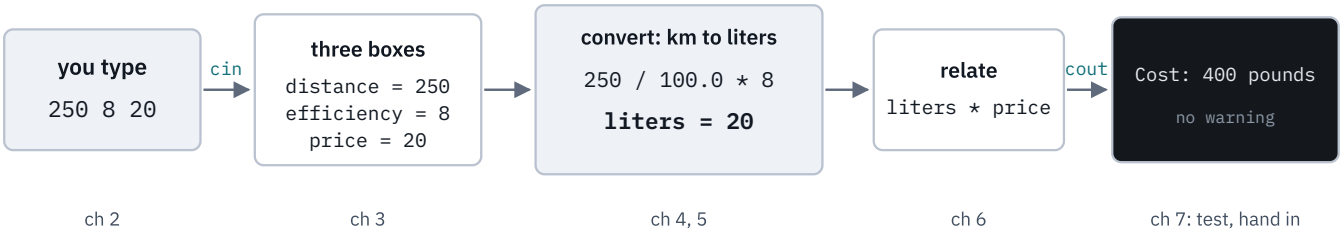
A program that asks you for its numbers can still give the wrong answer quietly

Sheet 2's question 8 asks what a car trip costs in fuel. You type the distance, the car's consumption and the price; the program prints the cost. With **250 km**, **8 liters per 100 km** and **20 pounds a liter** the answer is **400 pounds** - unless one variable is an `int`, and then the same program prints **320** without a single warning. This guide follows that trip from the file name to the zip you hand in.



What changes: Only the conversion box: distance is a double and the division is by 100.0, so 250 / 100.0 is 2.5.

What stays the same: The same keyboard, the same three numbers typed (250, 8, 20), the same cout.

The question it answers: What does the trip really cost? 400 pounds.

One box decides whether the answer is right. Everything you type, and everything printed, is identical in both settings. The numbers in the drawing are computed by this page from the same rule C++ uses; the compiled program prints 400 and 320.

Read it after the lab

Chapters follow the trip from left to right. Every number on the page was produced by compiling the code, on 4 October 2026.

The worksheet

One `.cpp` file with a step per chapter and the exercises left blank. Save it from the button at the top.

This week's questions

Sheet 2, Q1-Q6, Q8 and Q12 - the ones with no `if`. The other ten are next week's lab.

CHAPTER 1

Name it, save it: the professor’s rules

After this chapter you can make the sheet folder and every question file with exactly the names the professor specifies, on a lab PC, without help.

Name it › Read › Compute › Print › **Hand in**

1.1

Five rules to check before submission

Follow the professor's file and folder naming instructions for every sheet. The rules, from the note the professor sent to every student:

1. **No Arabic and no spaces** in any file or folder name. English letters, digits and underscores only.
2. **One folder per sheet**, named `XXX_Sheet_N`. `XXX` is your number on the class list, always three digits.
3. **One file per question**, named `Q_SSQQ.cpp`: two digits for the sheet, two for the question. Underscores, never dashes.
4. **Delete the `.exe` files** before you send. Dev-C++ makes one next to every `.cpp` you run.
5. **Archive the whole folder** into one file with the folder's name, and upload it at the link the professor announces. ZIP and RAR are both allowed; this guide uses Windows ZIP.

Consistent names let the marker identify your class-list number, sheet and question without opening each program. Do not substitute your university ID for your number on the class list.

1.2

Your names, worked out

Type your number on the class list - not your university ID, not the PC's number. The page builds the names the way the rules say.

FOLDER	FILE	ZIP TO SEND
034_Sheet_2	Q_0201.cpp	034_Sheet_2.zip

Number 34 becomes 034 (always three digits). Sheet 2, question 1 becomes 0201 (two digits each). The full path is Desktop\GEN241\034_Sheet_2\Q_0201.cpp.

The padding is the part people get wrong. Number 7 is `007`, question 1 is `01`. The names are computed by this page from the rules, as you type.

1.3

Check a name before you trust it

Type any file or folder name, or press one of the examples. Each one is a mistake that really happens in the lab.

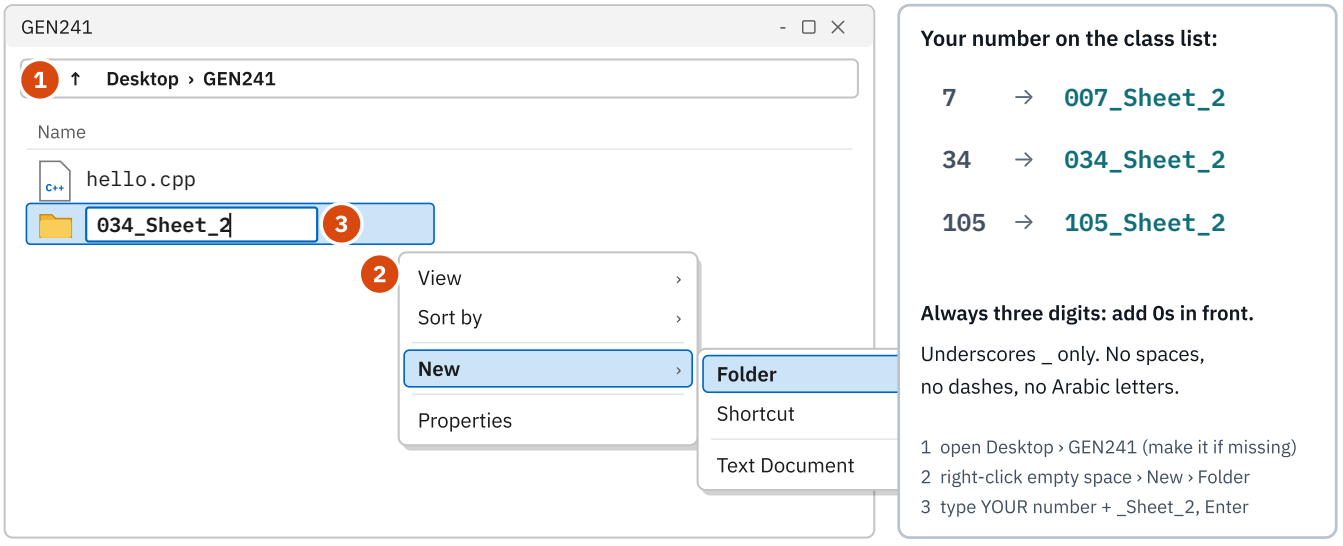
Follows the rules. A correct sheet folder name.

1.4

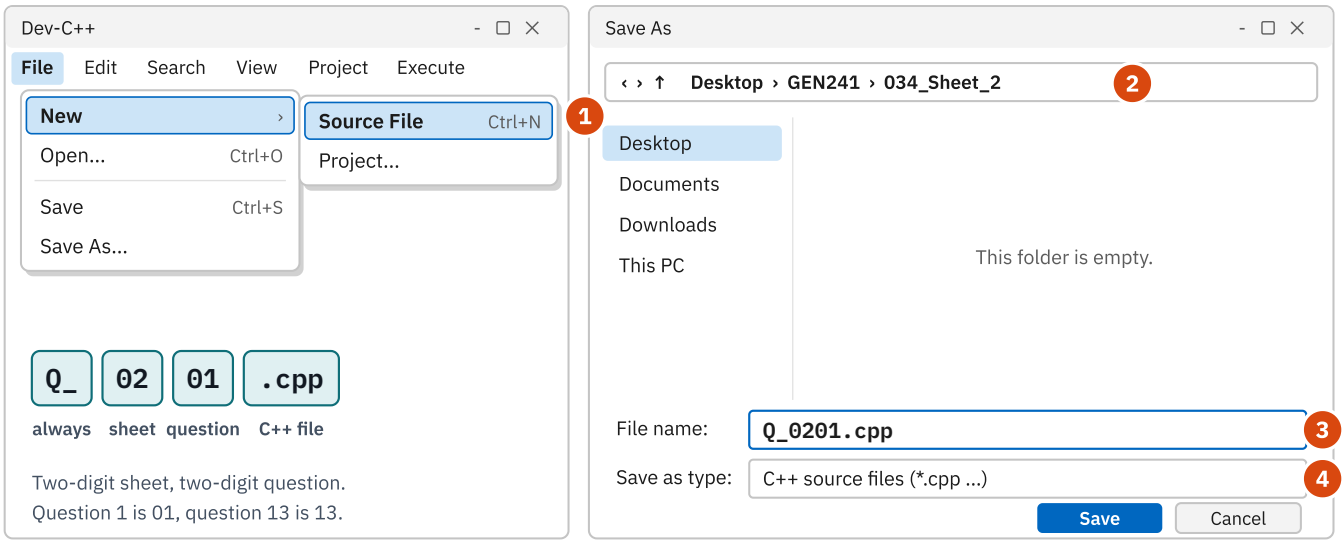
On a lab PC, step by step

The pictures are annotated illustrations of Windows 10. Windows 11 has the menu path given below.

1. Open `Desktop › GEN241` - the folder from week 1. If it is not there, make it: right-click the desktop, *New › Folder*, type `GEN241`.
2. Inside it, right-click an empty space, *New › Folder*, and type your folder name, e.g. `034_Sheet_2`. Press Enter.
3. Make sure *File name extensions* are switched on (Windows 10: *View › File name extensions*; Windows 11: *View › Show › File name extensions*), or `Q_0201.cpp.txt` looks like `Q_0201.cpp`.
4. In Dev-C++: *File › New › Source File*, then save it **before typing**: *File › Save*, go into your sheet folder, type `Q_0201.cpp`, and check that *Save as type* says C++.



The folder goes inside GEN241, named with your own number. Not 034 unless you are 034 on the list.



Save before you type. The file then lives in the sheet folder from its first line, with the name the professor specifies.

بالمصري

الرقم في الاسم هو رقمك في الكشف، دايمًا 3 أرقام: 7 تبقى 007. وسؤال 1 في شيت 2 يبقى Q_0201.cpp.
مفيش عربي ولا مسافات ولا شرطة.

WHAT COULD FOOL YOU

- **The wrong number.** Your university ID, the PC's number or your partner's number all make a valid-looking name that is still wrong.
- **Hidden extensions.** With extensions hidden, Windows shows `Q_0201.cpp.txt` as `Q_0201.cpp`. Switch them on once and it stays on.
- **Working inside Downloads.** A file opened from a WhatsApp download saves back to Downloads. Save it into your sheet folder first.

CHECK YOURSELF

You are number 105 on the list. Which is the file for sheet 2, question 8?

- A Q_028.cpp
- B 105_Q_0208.cpp
- C Q_0208.cpp
- D Q-0208.cpp

Back to the sheet: make `XXX_Sheet_2` now, and save every question into it as you go. Sheet 1 is handed in the same way: `XXX_Sheet_1` with `Q_0101.cpp` to `Q_0112.cpp`.

CHAPTER 2

cin: the program waits for you

After this chapter you can write a prompt and a `cin` for every input a question gives, and run the program with numbers you type.

Name it › **Read** › Compute › Print › Hand in

2.1

A prompt asks, cin reads

Every program on sheet 2 reads its numbers from the keyboard. Each input takes two lines: a `cout` that **asks** - it only prints text - and a `cin` that **reads**. When the program reaches `cin`, it stops and waits. Nothing happens until you type a number and press Enter.

sheet-02/Q_0208.cpp - the input part

```
    double distance, efficiency, price, liters, cost;

    cout << "Enter the distance to drive (km): ";
    cin >> distance;
    cout << "Enter the fuel efficiency (liters per 100 km): ";
    cin >> efficiency;
    cout << "Enter the price (pounds per liter): ";
    cin >> price;
```

CONSOLE - WHAT YOU SEE, TYPING 250, 8, 20

```
Enter the distance to drive (km): 250
Enter the fuel efficiency (liters per 100 km): 8
Enter the price (pounds per liter): 20
The fuel cost of the trip is 400 pounds
```

1. The bold numbers are what you typed. The rest was printed by the program. A program that only asks never gets an answer until you press Enter.
2. `cin >> distance;` reads a number and puts it in the box called `distance`. The arrows point the way the number travels: from the keyboard into the variable.
3. The prompt says the unit. A user who does not know the program wants kilometers will type miles.

2.2

Several numbers, one cin

One `cin` can read several numbers, in order: `cin >> distance >> efficiency >> price;`. You can type them on one line separated by spaces, or press Enter after each - cin does not care, it skips spaces and line breaks and takes the next number each time. This is how the lecture reads two values in its examples.

The order is the only thing that matters. Type `8 250 20` into the line above and the program believes the trip is 8 km and the car burns 250 liters per 100 km.

KEY IDEA

`cin` fills the variables in the order you wrote them, from whatever you type next, whatever the prompt said.

بالمصري

الـ cout بتسأل بس، والـ cin هي اللي بتقرا. البرنامج بيقف عند الـ cin ويستنى لحد ما تكتب رقم وتدوس Enter.

WHAT COULD FOOL YOU

- **"The program is frozen."** A console that just sits there after a prompt is waiting for you, not broken.
- **Commas.** Type `4 5`, not `4, 5` or `(4, 5)`. The sheet writes points with brackets; cin does not want them.
- **`cin << x`.** The arrows point from the keyboard into the variable: `cin >> x`. The wrong way round does not compile.

CHECK YOURSELF

The program runs `cin >> distance >> efficiency >> price;` and you type `250 8 20` on one line, then Enter. What happens?

A Only distance is filled; the other two wait for more typing

B distance = 250, efficiency = 8, price = 20

C An error - each number needs its own line

Back to the sheet: every week 2 question starts here. Q1 reads one number; Q4, Q5 and Q12 read points as pairs of numbers.

CHAPTER 3

How much a box holds: int and double

After this chapter you can choose `int` or `double` for every quantity in a question, and say what goes wrong if you choose badly.

Name it › **Read** › **Compute** › Print › Hand in

3.1

Whole numbers and numbers with a fraction

A variable is a box with a type, and the type decides what fits. An `int` holds whole numbers only. A `double` holds numbers with a fraction. Put 2.5 into an `int` and the .5 is simply dropped - no error, no warning on most compilers (lecture 3, page 8).

The question to ask for every quantity in a sheet question: **can it have a fraction?** A distance, a price, a temperature, a radius: yes, so `double`. A number of days, sides, people: no, so `int` - and then every division with it needs care (chapter 4).

Type	Holds	Size, lab PCs (Dev-C++)	Size, a Mac (clang)	Use it for
`int`	whole numbers, about ±2.1 billion	`4 bytes`	`4 bytes`	counts: days, sides, digits
`long`	whole numbers, at least as big as int	`4 bytes`	`8 bytes`	seconds since 1970 (Q16)

Type	Holds	Size, lab PCs (Dev-C++)	Size, a Mac (clang)	Use it for
`float`	fractions, about 7 digits	`4 bytes`	`4 bytes`	rarely - lecture 3 says prefer double
`double`	fractions, about 15 digits	`8 bytes`	`8 bytes`	almost every measured quantity

✓ CHECKED BY COMPILING IT

The Mac column was printed by `sizeof` with clang++ on 4 October 2026: 2, 4, 8, 8, 4, 8 bytes for short, int, long, long long, float, double. The lab column for `long` is the 4 bytes lecture 3's own screenshot shows (Ex_0302). `INT_MAX` printed 2147483647. Lecture 3's table says long is 8 bytes and its screenshot says 4: both are true, on different compilers. Run Ex_0302 on a lab PC to see your own.

3.2

The trip, typed

Distance, consumption and price can all have a fraction, so the trip uses `double` for every one of them. The spine at the top of the page shows the price of getting this wrong: make `distance` an `int`, divide it by `100`, and 250 km becomes 2 hundreds of km instead of 2.5. The fix is not a bigger type; it is the right one.

بالمصري

اسأل على كل كمية: ممكن يكون فيها كسر؟ لو آه يبقى double، لو لأ يبقى int. والـ int لو حطيت فيه كسر بيشيله من غير ما يقولك.

WHAT COULD FOOL YOU

- **"A bigger type fixes it."** `long` is still a whole number. 2.5 in a `long` is 2.
- **Copying `float` from a reference.** Some reference solutions use `float`. It works, with about 7 correct digits instead of 15; prefer `double`, as the lecture says.
- **Memorising sizes.** The sizes depend on the compiler. What does not change: `int` has no fraction, `double` does.

CHECK YOURSELF

Sheet 2, Q6: the number of sides of a polygon and the length of a side. Which types?

A int sides, int side length

B int sides, double side length

C double sides, double side length

Back to the sheet: Q2 (radius, length), Q4 and Q5 (coordinates) and Q8 are all `double`. Q3 (a number of days) is the one `int` question this week.

CHAPTER 4

/ and % do different jobs

After this chapter you can predict what any expression on sheet 2 prints before you run it, including every division and remainder.

`Name it` › `Read` › **`Compute`** › `Print` › `Hand in`

4.1

Two whole numbers, two answers

When both sides of `/` are whole numbers, C++ answers with a whole number: the quotient, with the fraction thrown away. `%` gives what was left over. Together they take a number apart, as lecture 3 does with 500 seconds: `500 / 60` is 8 whole minutes, `500 % 60` is the 20 seconds left.

Lecture 3's list, all checked by compiling them: `5 / 2` is 2, `-5 / 2` is -2 (toward zero, not down), `5.0 / 2` is 2.5, `26 % 8` is 2, `20 % 13` is 7, `12 % 4` is 0. When the left number is smaller, all of it is the remainder: `3 % 7` is 3.

4.2

Shortcuts that change a variable

`i += 2` means `i = i + 2`; the same works for `-=`, `*=`, `/=` and `%=`. `i++` adds one, `i--` subtracts one. Starting from `k = 10`: `k -= 4` gives 6, then `k *= 3` gives 18, then `k /= 4` gives 4 (whole numbers!), then `k %= 3` gives 1.

4.3

Try it: the calculator and C++ side by side

Type an expression, or press a preset. The left panel does every division the way a calculator does; the right one follows C++'s rule. When they differ, the page says exactly which division lost what. Predict before you press.

Expression `250 / 100 * 8`

A CALCULATOR SAYS

20

C++ PRINTS

16

Every operand was a whole number, so the result is an int and prints with no decimal part.

They differ. C++ reached `250/100`, found two whole numbers, and answered `2` instead of `2.5`. Give one operand a decimal point to fix it.

Both columns are computed by this page as you type, the right one following C++'s rules for int and double, `%` and `static_cast`. Every preset was also compiled and run; the page and the compiler agree.

4.4

Brackets decide the order

Inside an expression, `*`, `/` and `%` happen before `+` and `-` (lecture 3, operator precedence). So `y2 - y1 / x2 - x1` divides first. A fraction on the sheet needs brackets around its whole top and its whole bottom: Q5's slope is `(y2 - y1) / (x2 - x1)`, and Q6's area is `(n * s * s) / (4 * tan(PI / n))`.

KEY IDEA

Look at each `/` on its own: if both sides are whole numbers, the fraction is gone before anything else happens.

بالمصري

الـ / بين رقمين صحاح بتدّي الخارج بس، والـ % بتدّي الباقي. `500 / 60` يبقى 8 دقايق، و `500 % 60` يبقى 20 ثانية.

WHAT COULD FOOL YOU

- **"It rounds."** It never rounds: `9 / 5` is 1, not 2, and `-5 / 2` is -2.
- **A tidy answer.** 320, 132, 139.5: when a messy question gives a round-looking answer, check every division.
- `%` **with a fraction.** `7.5 % 2` does not compile; `%` is for whole numbers only.

CHECK YOURSELF

Q3 turns days into years: `numberOfDays / 365`, then `numberOfDays % 365`. For 500 days, what are the two results?

A 1.36986 and 135

B 1 and 135

C 1 and 1

Back to the sheet: Q3 is all `/` and `%`; Q5 and Q6 are all brackets.

CHAPTER 5

Where the decimal point has to go

After this chapter you can fix an integer division in the one place that fixes it, with a `.0` or a `static_cast`, and explain why storing the result in a `double` is too late.

Name it › Read › **Compute** › Print › Hand in

5.1

A double box does not save a whole-number division

Lecture 3's Ex_0307 averages 150 points over 16 items. It stores the answer in a `double` - and still prints 9:

lecture 3, Ex_0307 - the part that matters

```
1     int total_points = 150, items = 16;
2
3     double bad_avg = total_points / items;                      // 9
4     double good_avg = static_cast<double>(total_points) / items;  // 9.375
```

```
OUTPUT
Without cast: 9
With static_cast: 9.375
```

1. C++ works out the right-hand side first. `150 / 16` is two whole numbers, so the answer is 9 - and only then is 9 put into the double box. The box can hold 9.375; it never receives it.
2. `static_cast<double>(total_points)` turns 150 into 150.0 **before** the division, so the division is real.
3. Too late is still too late: `static_cast<double>(total_points / items)` casts the 9 after it was made, and prints 9.

5.2

The same trap, in Q1

Q1's formula is `fahrenheit = (9 / 5) * celsius + 32`, copied from the sheet. With `celsius = 100` it prints **132**, not 212: `9 / 5` is 1. The fix goes inside the division, where the fraction is lost: `9.0 / 5`. Test it with numbers you can check: 100 must give 212 and 37 must give 98.6.

5.3

The other way: double into int

Lecture 3 page 8: putting a double into an `int` drops the fraction, it does not round. `static_cast<int>(3.9)` is 3, and `static_cast<int>(-3.9)` is -3. That is what you want for "how many whole years" and never what you want for a price.

KEY IDEA

Put the decimal point, or the cast, on a number **inside** the division. Afterwards is too late.

✓ CHECKED BY COMPILING IT

9, 9.375 and 9 again for the late cast; 132 and 212 for Q1; 3 and -3 for the casts to int. clang++ -std=c++17 -Wall -Wextra, 4 October 2026. clang warns about `int i = 34.7;` (lecture 3's own example); Dev-C++ may compile it silently - the value is 34 either way.

بالمصري

الصندوق الـ double مش هينقذ قسمة صحيحة: الـ C++ بتحسب الأول وبعدين تحط النتيجة. حط الـ 0. جوه القسمة نفسها.

WHAT COULD FOOL YOU

- **"The variable is double, so it is fine."** The type of the box on the left never reaches back into the division on the right.
- **Fixing one division of two.** `15 + 30.0/60 + 30/3600` is still wrong: every `/` needs its own check.

CHECK YOURSELF

With `int minutes = 150;`, which line gives 2.5 hours?

A double hours = minutes / 60;

B double hours = static_cast<double>(minutes / 60);

C double hours = minutes / 60.0;

Back to the sheet: Q1 is this chapter. Check the division in Q8 too: the solution writes `distance / 100.0`.

CHAPTER 6

From the sheet's words to a program

After this chapter you can turn a paragraph like Q8 into five plan lines, a flowchart and a program whose every block comes from a plan line - and test it with a number you worked out first.

Name it › **Read** › **Compute** › **Print** › Hand in

6.1

Five lines before the keyboard

Q8: *read the distance to drive, the fuel efficiency of the car, and the price per liter, and display the fuel cost of the trip. Small cars are 6 to 12.* The sheet calls the efficiency "100 km per liter", but 6 to 12 only makes sense as **liters per 100 km** - 600 km on one liter is not a car - so that is how the solution reads it.

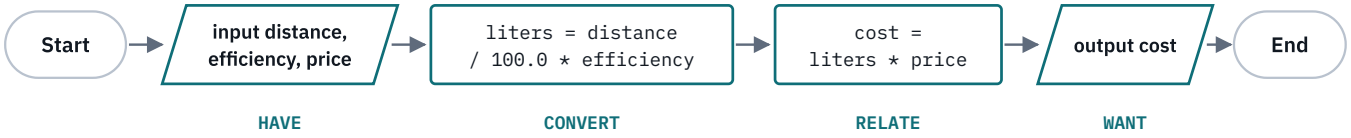
WANT	the fuel cost of the trip, in pounds
HAVE	distance in km · efficiency in liters per 100 km · price in pounds per liter
CONVERT	km to liters: `liters = distance / 100.0 * efficiency`
RELATE	`cost = liters * price`

CHECK	300 km at 8 L per 100 km is 24 L; at 20 pounds that is 480

6.2

The same plan as a flowchart

Lecture 3 draws programs with four shapes: an oval to start and end, a parallelogram for input and output, a rectangle for a calculation, and arrows for the order. Q8 has no decision, so its flowchart is one straight line - and each shape is one plan line.



Every shape is one plan line. The parallelograms are where the program talks to a person: `cin` in, `cout` out. CHECK is not a shape: it is the number you test with afterwards.

6.3

The program, block by block

sheet-02/Q_0208.cpp - the body of main, plan lines marked

```
    double distance, efficiency, price, liters, cost;          // HAVE

    cout << "Enter the distance to drive (km): ";
    cin >> distance;
    cout << "Enter the fuel efficiency (liters per 100 km): ";
    cin >> efficiency;
    cout << "Enter the price (pounds per liter): ";
    cin >> price;

    // 1. how much fuel the trip uses
    liters = distance / 100.0 * efficiency;                    // CONVERT

    // 2. what that fuel costs
    cost = liters * price;                                     // RELATE

    cout << "The fuel cost of the trip is " << cost << " pounds" << endl;   // WANT
```

TWO TESTS - THE SECOND CATCHES WHAT THE FIRST CANNOT

```
300 8 20  ->  The fuel cost of the trip is 480 pounds
250 8 20  ->  The fuel cost of the trip is 400 pounds
```

1. 480 is the CHECK line: the program agrees with the number you worked out by hand.
2. 300 cannot catch an integer division (300 / 100 is exactly 3). 250 can: written the int way it gives 320. Choose a test whose answer would change if the program were wrong.

6.4

Your turn: plan one alone

Not on the sheet: *a generator burns 4.5 liters of diesel per hour. It runs for 150 minutes. Diesel costs 15.5 pounds per liter. What does the run cost?* Write the five lines in your notebook first, without looking below. Then open the answer.

– **The five lines, and the trap** · open after you have written yours

WANT the cost of the run, in pounds. **HAVE** 4.5 L per hour, 150 minutes, 15.5 pounds per liter. **CONVERT** minutes to hours: `150 / 60.0` = 2.5 hours. **RELATE** liters = 4.5 × 2.5 = 11.25; cost = liters × 15.5. **CHECK** between the 2-hour and the 3-hour cost.

The program prints **174.375**. Written as `150 / 60` it prints **139.5**: two whole numbers, so 2 hours, and nothing warns you. Both numbers come from compiling it.

بالمصري

كل شكل في الـ `flowchart` هو سطر من الخطة، وكل جزء في الكود جه من سطر في الخطة. والـ CHECK هو الرقم اللي تجرّب بيه البرنامج بعد ما يخلص.

WHAT COULD FOOL YOU

- **Testing with friendly numbers.** 300 km, 60 minutes, 100 Celsius hide integer division. Test once with a number that does not divide evenly.
- **Skipping CHECK.** Without a number worked out first, 320 looks as believable as 400.

CHECK YOURSELF

Which test input would catch `int distance` with `distance / 100` in Q8?

A 200 km

B 300 km

C 150 km

Back to the sheet: Q8 is this chapter. Q12 is the same method with more steps: three distances (Q4's formula), then one area.

CHAPTER 7

This week's questions, and handing them in

The professor allows ZIP or RAR. We use Windows ZIP here. Two weeks from the 4 October message gives 18 October 2026; this is inferred, so follow the final announced deadline and upload link.

After this chapter you can test each of this week's questions against a known output and hand in a zip the professor's rules accept.

`Name it` › `Read` › `Compute` › **`Print`** › **`Hand in`**

7.1

Test every program with numbers you can check

Type the input in the third column. If your output differs, compare it with the right-hand column digit by digit before you change anything. The prompts may be worded differently in your program; the numbers may not.

Q	What it is testing	Type in	It prints
1	cin, and `9.0 / 5` (ch 2, 5)	`100 · then 37`	`212 · then 98.6`
2	a named constant PI (ch 3)	`5.5 12`	`area 95.0331 · volume 1140.4`
3	`/` and `%` on days (ch 4)	`500`	`1 years, 4 months and 15 days`
4	`pow(x, 0.5)` as a square root	`1.5 -3.4 · 4 5`	`8.76413`
5	brackets around a fraction (ch 4)	`4.5 -5.5 · 6.6 -6.9`	`-0.666667`
6	`int` sides, `tan` in radians	`4 · 2`	`4 (a 2 × 2 square)`
8	the trip: plan, convert, test (ch 6)	`250 · 8 · 20`	`400 pounds`
12	Q4 three times, then one area	`0 0 4 0 0 3`	`6`

✓ CHECKED BY COMPILING IT

Every output in the table was printed by the solution files, compiled with clang++ -std=c++17 -Wall -Wextra on 4 October 2026 and run with exactly that input. Q6 with 4 sides of 2 is a square, so 4 is checkable by hand; Q12's points make a right triangle with sides 3 and 4, so 6 is too.

7.2

Where things stand

SETTLED

The language behaviour in chapters 2 to 6

Integer division, `%`, the casts and cin's order are defined by C++ and behave the same in Dev-C++ on the lab PCs and clang++ on a Mac. Only the sizes in chapter 3 differ between compilers.

OPEN · Q3

Months or weeks

The sheet asks for years, months and days but gives no month length. This guide uses 30-day months. **What it changes:** two lines. Your TA will confirm with the professor.

READ THIS WAY · Q8

Liters per 100 km

The sheet's words say "100 km per liter", its numbers (6 to 12) say liters per 100 km, and so does the professor's own solution. **What it changes:** the formula, completely.

7.3

Before you zip it

Ticks are kept in this browser only. Zip at the very end: a zip is a copy of the folder at that moment, and editing a `.cpp` afterwards does not change it.

- [ ] **The folder is your number, three digits, _Sheet_2**
  034_Sheet_2, inside Desktop › GEN241. Chapter 1's builder makes it for you.
- [ ] **Every file is Q_02 + two digits + .cpp**
  Q_0201.cpp, Q_0208.cpp, Q_0212.cpp. Extensions switched on, so .cpp.txt cannot hide.
- [ ] **Every file compiles after your last edit**
  Compile each one again. "It worked yesterday" is not a test.
- [ ] **Every program matches the table in 7.1**
  Same input, same numbers.

- [ ] **Every division checked on its own**
  Both sides whole numbers? Did you mean that?

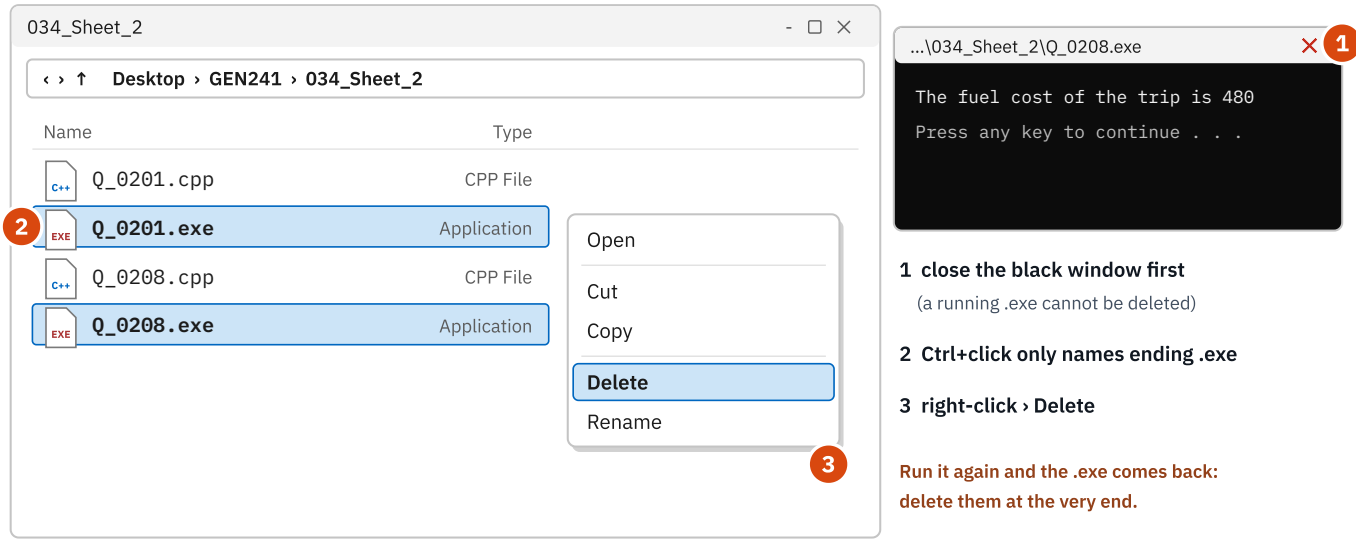
- [ ] **The .exe files are deleted**
  Close the black window first. Delete only names ending .exe.

- [ ] **The zip holds one folder, named like the zip**
  Zip from GEN241, not from inside the folder. Open the zip and look.

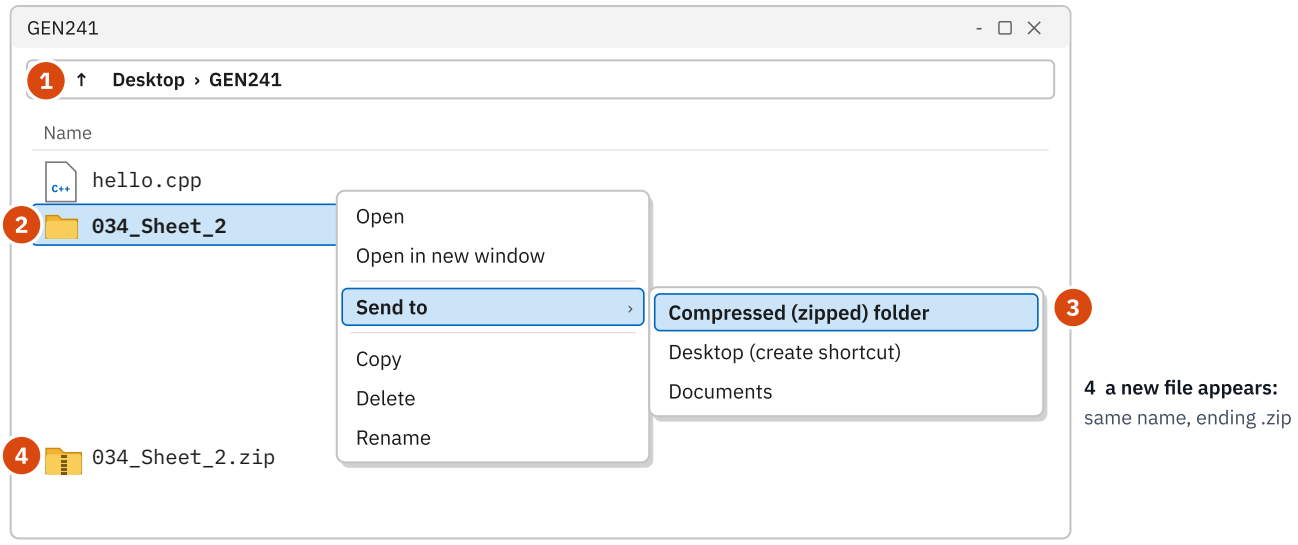
7.4

Packaging on a lab PC, picture by picture

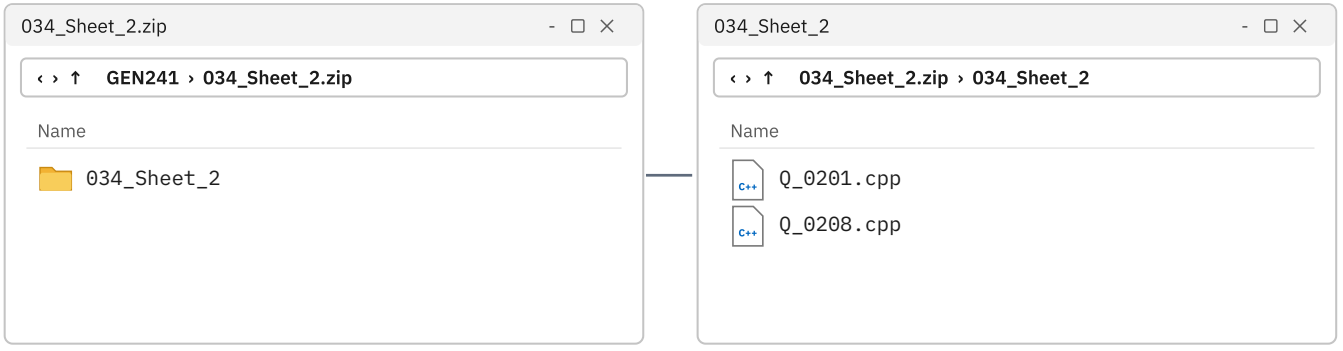
Windows 11: right-click the folder, then Show more options › Send to › Compressed (zipped) folder. Windows 10 uses Send to directly. Do this at the very end, after the last test run. The same three moves package sheet 1 into `XXX_Sheet_1`.



Close the black window first. Windows will not delete an `.exe` that is still running. Delete only the names ending in `.exe`.



Zip the folder, from one level up. Zipping the files inside it gives a zip with no folder in it.



- ✓ the zip holds ONE folder, named like the zip
- ✓ inside it: only .cpp files, no .exe
- ✓ every file is Q_02 + two digits + .cpp

Open your own zip before you upload it. One folder, named like the zip, holding only `.cpp` files with correct names.

– Next week · lab 3, when selection syntax is taught

The other ten questions - Q7, Q9-Q11 and Q13-Q18 - make decisions: is the triangle valid, is the point inside the circle, which BMI class. They need `if` and `else`, introduced at the end of lecture 3 and developed further in Liang chapter 3. Sheet 2 is due two weeks after it was given, so both labs count toward it. Keep everything in the same `XXX_Sheet_2` folder.

Sources and checks

- **Sheet (2)**, Computer Languages & Applications, Prof. Mahmoud M. Khater: 18 questions, due two weeks after 4 October 2026. Q1-Q6, Q8 and Q12 are this guide's; the rest are week 3's.
- **Lecture 3** (GEN_241_03, 4 October 2026): numeric types and sizes, `/` and `%`, `pow`, precedence, augmented assignment, `++`/`--`, conversions and `static_cast`, flowcharts. Ex_0302 and Ex_0307 are quoted from it.
- **The professor's file and folder rules**, sent to students as "File and Folder Names" (PDF).
- **Every output on this page was produced by compiling and running code** with clang++ -std=c++17 -Wall -Wextra on a Mac, 4 October 2026. Sizes on the lab PCs come from lecture 3's own screenshot.
- The trip's numbers are the solution file's own tests; the generator problem was written for teaching and is not on the sheet.

For GEN 241 students after the week 2 lab. It covers the questions with no `if`; selection is week 3.